Boson14 Planted-Clique Benchmark Suite

Scaled Motzkin-Straus Calibration for Dirac-3

QCi Hardware Team

April 8, 2026

Quantum Computing Inc.

Why Planted Cliques?

- **Known** ω — ground truth is planted, not discovered
- **Controllable difficulty** — adjust n, k, edge probability p
- **Integer coupling** — $J = -A$ has entries in $\{0, -1\}$, no half-integer scaling
- **Degenerate landscapes** — plant multiple same-size cliques to test multi-modal behaviour
- **Blind testing** — scramble vertex labels; solver sees no hint of planted structure

Key insight: If hardware recovers the planted ω across different R values and permutations, calibration is verified end-to-end.

Motzkin-Straus Theorem

The clique number can be found via a continuous optimisation:

Theorem (Motzkin & Straus, 1965)

$$\max_{x \in \Delta_n} \frac{1}{2} x^\top A x = \frac{1}{2}\left(1 - \frac{1}{\omega}\right)$$

where A is the adjacency matrix and $\Delta_n = \{x \geq 0 : \sum x_i = 1\}$ is the standard simplex.

Inverting: Given optimal objective f^*:

$$\omega = \text{round}\left(\frac{1}{1 - 2f^*}\right)$$

Scaled Simplex Formulation

Scale to integer-friendly constraints: $y = R\,x$, so $\sum y_i = R$.

$$g(y) = \tfrac{1}{2}\, y^\top A\, y = R^2 \cdot f(x)$$

$$g^*(\omega) = \frac{R^2}{2}\left(1 - \frac{1}{\omega}\right)$$

$$\omega = \text{round}\left(\frac{R^2}{R^2 - 2\,g^*}\right)$$

- Optimal allocation: $y_i = R/\omega$ for clique vertices, 0 otherwise
- Coupling matrix: $J = -A$ (entries in $\{0, -1\}$, all integers)

Hardware Energy Mapping

Dirac-3 **minimises** $E(y) = y^\top J\, y$ on the scaled simplex $\sum y_i = R$.

$$E(y) = y^\top J\, y = -y^\top A\, y = -2\, g(y)$$

$$E^* = -R^2\left(1 - \frac{1}{\omega}\right)$$

$$\omega = \mathrm{round}\left(\frac{R^2}{R^2 + E^*}\right)$$

- Both $g \to \omega$ and $E \to \omega$ inversions should agree on every sample
- Consistency check: count agreement rate across all Dirac-3 samples

Toolkit File Structure

```
boson14/
  generate.py                # Instance generation (planted, degenerate, scramble)
  test_dirac.py              # Dirac-3 submission + verification
  scaled-motzkin-straus.md   # Mathematical derivation
  USAGE.md                   # Full CLI reference
  manual.pdf                 # This presentation
  inputs/                    # External CSV files (e.g. DIMACS graphs)
    C125.9.csv               # 125-vertex dense graph (omega=34)
  output/                    # Results directory
    *_meta.json              # Instance metadata
    *_boson14.csv            # StQP adjacency CSV (n x n+1)
    *_dirac_meta.json        # Solver results
    *_solutions.npz          # Theoretical solutions
    *_dirac_solutions.npz    # Hardware samples
    *.png                    # Plots
```


Library: Core functions live in `dirac_bench.boson14` (the `dirac-test-suite`

generate.py — Generating Instances

```
uv run python boson14/generate.py --n 30 --k 12 --verify
```

```
Generating: n=30, k=12, p=0.5, seed=42, R=100
  Planted nodes: [2, 3, 6, 10, 14, 16, 19, 21, 23, 28, 29, 30]

Theoretical values (R=100, w=12):
  g* = 4583.33
  E* = -9166.67
  y_i = R/w = 100/12 = 8.3333 for planted vertices

Integer QP: J has entries in {-1, 0}

Brute-force verification:
  Clique number (brute-force): w = 12
  CONFIRMED: w = k = 12
```

Variants:

```
# Dense edges, custom seed
generate.py --n 30 --k 12 --density dense --seed 99
```

Degenerate Cliques

Plant multiple same-size cliques to create a **degenerate** optimisation landscape:

- All planted cliques have size k — same ω, multiple optima
- Cross-edges between planted sets are **severed** to guarantee independence
- Tests whether hardware explores multiple basins or collapses to one

```
uv run python boson14/generate.py --n 50 --k 10 --num-cliques 3 --verify
```

```
Degenerate: 3 planted 10-cliques
Set 1: [6, 18, 21, 26, 28, 40, 46, 47, 48, 50]
Set 2: [5, 8, 19, 24, 25, 30, 32, 38, 39, 41]
Set 3: [10, 16, 17, 22, 27, 29, 33, 35, 42, 43]
omega_verified: 10, num_max_cliques: 3
```

Index Scrambling

Apply a random vertex permutation for blind testing:

- **Forward permutation:** original label $\rightarrow$ scrambled label
- **Inverse permutation:** scrambled label $\rightarrow$ original label (stored in metadata)
- `unscramble_solution()` maps Dirac-3 y-vectors back to original ordering
- Objectives are invariant: $g(y_{\text{scrambled}}) = g(y_{\text{original}})$

```
# Generate scrambled
uv run python boson14/generate.py --n 50 --k 15 --num-cliques 3 --scramble
# Test scramble roundtrip on Dirac-3
uv run python boson14/test_dirac.py --n 14 --k 7 --scramble --verify --plot
```

```
--- Scramble Roundtrip ---
  Sample 0: g(scrambled)=4285.65, sum(y_orig)=100.00
  Scramble roundtrip: objectives preserved through permutation
```

test_dirac.py — Dirac-3 Submission

Three input modes: inline parameters, JSON from `generate.py`, or CSV file:

```
# Inline planted-clique parameters
uv run python boson14/test_dirac.py --n 14 --k 7 --verify --plot
# From generate.py JSON
uv run python boson14/test_dirac.py --from-json .../boson14.json --verify
# From CSV (planted or external graph)
uv run python boson14/test_dirac.py --csv .../boson14.csv --k 7 --verify
```

```
============================================================
Boson14 Dirac-3 Test: n=14, k=7, p=0.5, seed=42, R=100
============================================================
Planted nodes: [3, 5, 6, 7, 8, 12, 13]

Theoretical values (R=100, w=7):
  g* = 4285.71,  E* = -8571.43,  y_i = 14.2857

--- Dirac-3 Submission (R=100) ---
  Submitting 14-variable scaled QP to Dirac-3 cloud
```

CSV Input Path

Load any graph from a CSV adjacency file — no planted-clique generation required.

CSV format: n rows $\times$ $(n{+}1)$ columns. Column 0 = linear coefficients C_i (all zeros for Motzkin-Straus), columns $1 \ldots n$ = symmetric adjacency A.

With known k (planted-clique CSV from `generate.py`):

```
uv run python boson14/test_dirac.py \
  --csv boson14/output/n14_k7_p0.5_s42/n14_k7_p0.5_s42_boson14.csv \
  --k 7 --verify --plot
```

External graph, unknown k (e.g. DIMACS):

```
uv run python boson14/test_dirac.py \
  --csv boson14/inputs/C125.9.csv --verify
```

- CSV→JSON round-trip is verified automatically (adjacency survives conversion)

CSV Input — Sample Output

```
Loaded CSV: .../n14_k7_p0.5_s42_boson14.csv (n=14)
  CSV->JSON round-trip: PASS

============================================================
Boson14 Dirac-3 Test: n=14, k=7, R=100  (from CSV)
============================================================

Theoretical values (R=100, w=7):
  g* = 4285.71,  E* = -8571.43,  y_i = 14.2857

Brute-force verification:
  Clique number (brute-force): w = 7
  CONFIRMED: w = k = 7

--- Dirac-3 Submission (R=100) ---
  Best g = 4285.65  =>  omega = 7  (161.4s)

Conversion consistency:
  omega_from_g == omega_from_E: 100/100 samples agree
```

Multi-R Sweep

Verify that omega recovery is consistent across different R values:

```
uv run python boson14/test_dirac.py --n 14 --k 7 --multi-R --verify
```

```
--- Multi-R Sweep ---

      R |         g* |         E* |     best_g | best_omega
  -----+-----------+-----------+-----------+-----------
      1 |     0.4286 |    -0.8571 |     0.4286 |          7
     10 |    42.8571 |   -85.7143 |    42.8571 |          7
    100 |  4285.7143 | -8571.4286 |  4285.6501 |          7

  PASS: All R values recover omega = 7
```

- Tests $R \in \{1, 10, 100\}$ on the same graph instance
- All three must agree on ω for the test to PASS
- Validates that energy scaling does not affect clique recovery

Generating Distribution Plots

Two plot types are available:

Clique distribution (`clique_dist_*.png`) — two-panel figure: top panel shows maximal clique counts by size, bottom panel shows the scaled objective g^* for each maximal clique with vertical ω lines.

Scaled histogram (`scaled_histogram_*.png`) — histogram of Dirac-3 sample objectives with theoretical $g^*(\omega)$ reference lines.

From generate.py (local brute-force, no hardware needed):

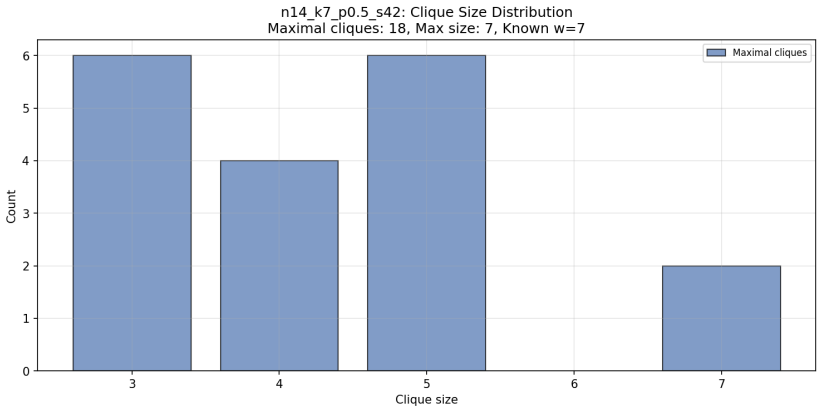
```
uv run python boson14/generate.py --n 14 --k 7 --plot-distribution --verify
```

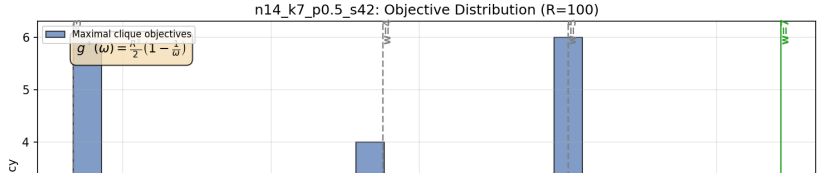
From test_dirac.py (both plot types, after Dirac-3 submission):

```
uv run python boson14/test_dirac.py --n 14 --k 7 --verify --plot
```

Plots are saved to `--output-dir` (default: `boson14/output/`).

Example: Clique Distribution (n=14, k=7)





Scaled Histogram — What It Shows

The histogram plots the distribution of $g(y) = \frac{1}{2} y^\top A y$ across all Dirac-3 samples:

- **Vertical lines** mark theoretical $g^*(\omega)$ for integer ω values
- **Green line:** known ω (planted clique size)
- **Red line:** best objective achieved by hardware
- **Histogram shape:** indicates consistency of hardware output

Tight clustering near the planted ω line = well-calibrated hardware.

$$g^*(\omega) = \frac{R^2}{2}\left(1 - \frac{1}{\omega}\right)$$

Interpreting Results

What histograms tell you about hardware quality:

Peak at planted ω: Hardware finds optimal solutions consistently.

Peak 1–2 below planted ω: Good performance, slight calibration drift.

Wide spread: Hardware output is noisy; may need parameter tuning.

Bimodal distribution: Hardware may be finding multiple local optima (expected for degenerate instances).

Tuning knobs:

- `relaxation_schedule`: 1–4, controls annealing behaviour
- `num_samples`: More samples $\rightarrow$ better statistics
- `R`: Larger $R \rightarrow$ larger energy gap between clique sizes

Output Formats

Suffix	Source	Contents
`_meta.json`	generate.py	Graph params, planted sets, g^*, E^*
`_dirac_meta.json`	test_dirac.py	Solver params, best ω, solve time
`_solutions.npz`	both	Theoretical max-clique y-vectors
`_dirac_solutions.npz`	test_dirac.py	Dirac-3 sample y-vectors
`clique_dist_*.png`	both	Clique size distribution bar chart
`scaled_histogram_*.png`	test_dirac.py	Objective histogram

Loading solutions in Python:

```python
import numpy as np
data = np.load("boson14/output/n14_k7_p0.5_s42_dirac_solutions.npz")
y_vectors = data["y_vectors"]  # shape (num_samples, n)
```

Worked Example ($R = 100$, $\omega = 7$)

Quantity	Formula	Value
f^*	$\frac{1}{2}(1 - 1/7)$	0.4286
g^*	$\frac{100^2}{2}(1 - 1/7)$	4285.71
E^*	$-100^2(1 - 1/7)$	-8571.43
y_i (clique)	$100/7$	14.2857
y_i (non-clique)	0	0

Verification:

- $\omega = \text{round}(10000/(10000 - 8571.43)) = \text{round}(7) = 7$ ✓
- $\omega = \text{round}(10000/(10000 + (-8571.43))) = \text{round}(7) = 7$ ✓
- $\sum y_i = 7 \times 14.2857 = 100 = R$ ✓